

Môn: Thống kê máy tính và ứng dụng

GVHD: Trương Vĩnh Linh

Ôn tập TH GK- 10/20224

Phúc Lâm – Thời gian làm bài: 60p

Nội dung thi giữa kì

- Thống kê mô tả (tính toán các độ đo, trực quan hóa dữ liệu)\
- Mô phỏng
- Phân phối xác suất

Lưu ý:

- Thi tại máy tính trường, không dùng laptop cá nhân, không internet
- Các hình thức gian lận hoặc dùng bot đều sẽ không được chấm điểm
- Sinh viên cần đến sớm để chuẩn bị máy, cài đặt thư viện
- Chỉ được sử dụng tài liệu giấy, không sử dụng tài liệu mềm

Luyện tập:

- Xem lại các bài lab cũ
- Tham khảo các bài viết chủ đề **statistics** trên kaggle

Đề 01 – sử dụng bộ dữ liệu brfss_2000.csv

Hệ thống giám sát rủi ro dựa trên hành vi (The Behavior Risk Factor Surveillance System - BRFSS) là một cuộc khảo sát qua điện thoại hàng năm với 350.000 người ở Hoa Kỳ. Như tên gọi của nó, BRFSS được thiết kế để xác định các yếu tố nguy cơ ở người trưởng thành và báo cáo các xu hướng sức khỏe mới. Ví dụ, người trả lời được hỏi về chế độ ăn uống và hoạt động thể chất hàng tuần, tình trạng HIV/AIDS, khả năng sử dụng thuốc lá và thậm chí cả mức độ chi trả dịch vụ chăm sóc sức khỏe của họ.

Bộ dữ liệu **brfss_2000** chứa thông tin khảo sát năm 2000, với hơn 200 thông tin. Trong bộ dữ liệu này, ta chỉ khảo sát một số thông tin sau:

- **genhlth**: người khảo sát tự đánh giá sức khỏe (excellent, very good, good, fair or poor)
- **exerany**: cho biết có hoạt động thể chất nào trong tháng gần nhất hay không, có (1), không (0)
- **hlthplan**: có bảo hiểm (1) hay không (0)
- **smoke100**: tổng số điều thuốc đã hút ít nhất
- **height**: chiều cao (inches)
- **weight**: cân nặng (pounds)
- **wt desire**: cân nặng mong muốn (pounds)
- **age**: tuổi
- **gender**: giới tính: nam(m), nữ(f)

Câu 1: Đọc hai bộ dữ liệu trên và cho biết mỗi bộ dữ liệu có kích thước bao nhiêu dòng, bao nhiêu cột?

```
import pandas as pd
#đọc bộ dữ liệu
data = pd.read_csv('brfss_2000.csv')
print("Kích thước dữ liệu: ",data.shape) #số dòng, số cột
print("Số dòng:",data.shape[0])
print("Số cột:",data.shape[1])
print(data.head(10))          # Hiển thị 10 dòng dữ liệu đầu tiên
```

Kích thước dữ liệu: (20000, 9)

Câu 2

a, Tính tỷ lệ nam nữ

```
gender_counts = data['gender'].value_counts(normalize=True)
print("Tỷ lệ nam nữ:\n", gender_counts)
```

Tỷ lệ nam nữ:

```
gender
f      0.52155
m      0.47845
```

Name: proportion, dtype: float64

b, Trong số những người tập thể thao, tỷ lệ những người tự đánh giá có sức khỏe kém là bao nhiêu?

```
exercise_data = data[data['exerany'] == 1]
poor_health_rate = exercise_data[exercise_data['genhlth'] == 'poor'].shape[0] /
exercise_data.shape[0]
print("Tỷ lệ sức khỏe kém trong nhóm tập thể thao:", poor_health_rate)
```

Tỷ lệ sức khỏe kém trong nhóm tập thể thao: 0.01964597022931474

Câu 3

a, Đổi đơn vị chiều cao từ inches sang centimet, đơn vị cân nặng từ pound sang kg.

```
data['height_cm'] = data['height'] * 2.54
data['weight_kg'] = data['weight'] * 0.453592
```

Kiểm tra kết quả

```
print(data[['height_cm', 'weight_kg']].head())
```

	height_cm	weight_kg
0	177.80	79.378600
1	162.56	56.699000
2	152.40	47.627160
3	167.64	59.874144
4	154.94	68.038800

b, Tính tỷ lệ những người muốn giảm cân.

```
weight_loss_data = data[data['wtdesired'] < data['weight']]
weight_loss_rate = weight_loss_data.shape[0] / data.shape[0]
print("Tỷ lệ người muốn giảm cân:", weight_loss_rate)
```

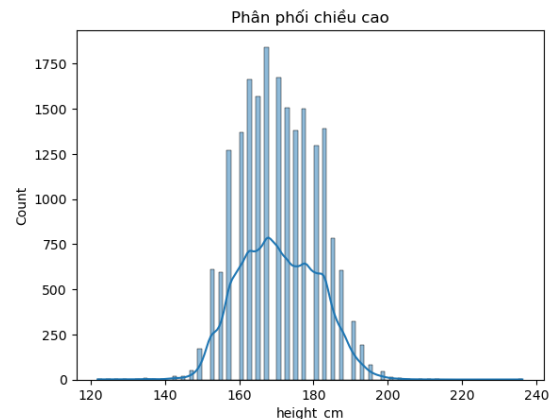
Tỷ lệ người muốn giảm cân: 0.6382

Câu 4:

a, Theo bạn trong các thuộc tính trên, thuộc tính nào có phân phối chuẩn. Vẽ hình minh họa

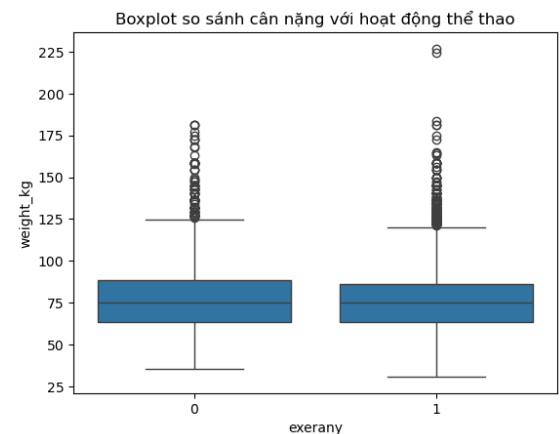
```
import matplotlib.pyplot as plt
import seaborn as sns

# Vẽ histogram để kiểm tra phân phối
sns.histplot(data['height_cm'], kde=True)
plt.title("Phân phối chiều cao")
plt.show()
```



b, Vẽ đồ thị boxplot so sánh cân nặng của những người có tập thể dục

```
#b. Vẽ boxplot so sánh cân nặng của những người có tập thể dục
sns.boxplot(x='exerany', y='weight_kg', data=data)
plt.title("Boxplot so sánh cân nặng với hoạt động thể thao")
plt.show()
```



Câu 5

Phân bố tuổi tác trong mẫu: Hãy mô tả phân bố tuổi của người tham gia khảo sát.

Tuổi trung bình, độ lệch chuẩn, và các phân vị 25%, 50%, 75% là bao nhiêu?

```
age_stats = data['age'].describe()
print("Tuổi trung bình:", age_stats['mean'])
print("Độ lệch chuẩn:", age_stats['std'])
print("Phân vị 25%:", age_stats['25%'])
print("Phân vị 50%:", age_stats['50%'])
print("Phân vị 75%:", age_stats['75%'])
```

```
Tuổi trung bình: 45.06825
Độ lệch chuẩn: 17.19268947448475
Phân vị 25%: 31.0
Phân vị 50%: 43.0
Phân vị 75%: 57.0
```

Câu 6

Tỉ lệ người hút thuốc: Tính tỉ lệ phần trăm người tham gia khảo sát hiện đang hút thuốc lá. Liệu có sự khác biệt đáng kể về tỉ lệ này giữa các nhóm tuổi khác nhau không?

```
smoking_rate = data[data['smoke100'] > 0].shape[0] / data.shape[0]
print("Tỷ lệ hút thuốc:", smoking_rate)

# So sánh tỷ lệ giữa các nhóm tuổi
age_groups = pd.cut(data['age'], bins=[18, 30, 45, 60, 80, 100])
smoking_by_age = data.groupby(age_groups)['smoke100'].apply(lambda x: (x > 0).mean())
print(smoking_by_age)
```

```
Tỷ lệ hút thuốc: 0.47205
age
(18, 30]      0.394535
(30, 45]      0.449946
(45, 60]      0.529298
(60, 80]      0.553034
(80, 100]     0.421811
```

```

from scipy.stats import chi2_contingency

# Tạo bảng tần suất cắt ngang giữa các nhóm tuổi và việc hút thuốc
contingency_table = pd.crosstab(age_groups, data_cleaned['smoke100'] > 0)

# Thực hiện kiểm định Chi-Square
chi2, p, dof, expected = chi2_contingency(contingency_table)

print("Kiểm định Chi-Square")
print(f"Chi2 statistic: {chi2}, p-value: {p}")

if p < 0.05:
    print("Có sự khác biệt đáng kể về tỷ lệ hút thuốc giữa các nhóm tuổi.")
else:
    print("Không có sự khác biệt đáng kể về tỷ lệ hút thuốc giữa các nhóm tuổi.")

```

```

Kiểm định Chi-Square
Chi2 statistic: 281.16304169612334, p-value: 1.2509150316633166e-59
Có sự khác biệt đáng kể về tỷ lệ hút thuốc giữa các nhóm tuổi.

```

Câu 7

BMI trung bình theo giới tính: Tính chỉ số BMI trung bình cho nam và nữ trong mẫu.

```

data['BMI'] = data['weight_kg'] / (data['height_cm']/100)**2
bmi_by_gender = data.groupby('gender')['BMI'].mean()
print("BMI trung bình theo giới tính:\n", bmi_by_gender)

```

```

BMI trung bình theo giới tính:
gender
f    25.743674
m    26.926313
Name: BMI, dtype: float64

```

Câu 8

Phân tích tỉ lệ bệnh béo phì: Sử dụng các tiêu chuẩn của CDC về chỉ số BMI để phân loại người tham gia vào nhóm béo phì. Tính tỉ lệ béo phì theo giới tính và độ tuổi.

```
data['obese'] = data['BMI'] >= 30

# Tỷ lệ béo phì theo giới tính
obesity_by_gender = data.groupby('gender')['obese'].mean()
print("Tỷ lệ béo phì theo giới tính:\n", obesity_by_gender)

# Tỷ lệ béo phì theo độ tuổi
obesity_by_age = data.groupby('age_groups')['obese'].mean()
print("Tỷ lệ béo phì theo độ tuổi:\n", obesity_by_age)
```

```
Tỷ lệ béo phì theo giới tính:
gender
f      0.185121
m      0.201170
Name: obese, dtype: float64
Tỷ lệ béo phì theo độ tuổi:
age
18-29      0.141599
30-44      0.209964
45-59      0.231380
60-79      0.198742
80+        0.088477
Name: obese, dtype: float64
```

--- Kết thúc đề 01 ---

ĐỀ 02 – SỬ DỤNG BẢNG DỮ LIỆU STUDENTSURVERY.CSV

Đề bài: DỮ LIỆU SINH VIÊN

Dữ liệu được cung cấp trong file StudentSurvey.csv được khảo sát. Yêu cầu

Câu 1.

- (1 điểm) Hiển thị 5 dòng dữ liệu
- (1 điểm) Xây dựng bảng phân phối tần số thể hiện số lượng sinh viên mỗi năm, bỏ qua các dòng bị thiếu dữ liệu năm.

Câu 2. Dựa vào dữ liệu bạn hãy cho biết:

- a. (2 điểm) Tỷ lệ sinh viên đeo khuyên?
- b. (1 điểm) Giữa sinh viên nam và nữ tỷ lệ đeo khuyên của ai cao hơn?

Câu 3 .

- (1.5 điểm) Vẽ đồ thị histogram của ba thuộc tính nhịp tim (Pulse), số giờ tập thể thao 3 (Exercise), số khuyên đeo (Piercings).
- (0.5 điểm) Hãy cho biết phân phối của ba thuộc tính trên phân phối nào lệch trái, lệch phải hoặc đối xứng.

Câu 4. (3 điểm) Sử dụng hàm sample() trên DataFrame. Bạn hãy:

- (1 điểm) Lấy ngẫu nhiên một mẫu từ bộ dữ liệu trên với cỡ mẫu là 50. Tính tỷ lệ nữ mẫu trên. Tính sai số so với giá trị tỷ lệ từ bộ dữ liệu ban đầu.
- (1 điểm) Hãy thực hiện lại câu trên 100 lần với cùng cỡ mẫu là 50, với mỗi mẫu hãy tính giá trị tỷ lệ nữ của mẫu và vẽ histogram của 100 giá trị trung bình này
- (1 điểm) Lập lại câu b và thử nghiệm với các cỡ mẫu khác nhau. Bạn có nhận thấy có gì khác biệt không? Nếu có hãy giải thích

Year	Year in school: First Year, Sophomore, Junior, or Senior
Gender	Student's gender: F or M
Smoke	Smoker? No or Yes
Award	Preferred award: Academy, Nobel, or Olympic
HigherSAT	Which SAT is higher? Math or Verbal
Exercise	Hours of exercise per week
TV	Hours of TV viewing per week
Height	Height (in inches)
Weight	Weight (in pounds)
Siblings	Number of siblings
BirthOrder	Birth order, 1 oldest, 2 second oldest, etc.
VerbalSAT	Verbal SAT score
MathSAT	Math SAT score
SAT	Combined Verbal + Math SAT
GPA	Grade point average
Pulse	Pulse rate (beats per asimate)
Piercings	Number of body piercings

```
import pandas as pd
import matplotlib.pyplot as plt
```

Câu 1:

1. Hiển thị 5 dòng dữ liệu:

```
# Đọc dữ liệu
data = pd.read_csv('StudentSurvey.csv')

# Hiển thị 5 dòng dữ liệu đầu tiên
print(data.head())
```

2. Xây dựng bảng phân phối tần số thể hiện số lượng sinh viên mỗi năm, bỏ qua các dòng bị thiếu dữ liệu năm:

```
# Loại bỏ các dòng bị thiếu dữ liệu trong cột Year
data_cleaned = data.dropna(subset=['Year'])

# Tạo bảng phân phối tần số của cột Year
year_distribution = data_cleaned['Year'].value_counts()

# Hiển thị bảng phân phối tần số
print(year_distribution)
```

Câu 2:

a. Tỷ lệ sinh viên đeo khuyên

```
# Tính tỷ lệ sinh viên đeo khuyên (Piercings > 0)
piercing_rate = (data['Piercings'] > 0).mean() * 100
print(f"Tỷ lệ sinh viên đeo khuyên: {piercing_rate:.2f}%")
```

b. Tỷ lệ đeo khuyên giữa sinh viên nam và nữ:

```
piercing_rate_gender = data.groupby('Gender')['Piercings'].apply(lambda x:
(x > 0).mean() * 100)
print(f"Tỷ lệ đeo khuyên theo giới tính:\n{piercing_rate_gender}")
```

Câu 3:

a. Vẽ biểu đồ histogram cho nhịp tim (Pulse), số giờ tập thể thao (Exercise), và số khuyên đeo (Piercings):

```
import matplotlib.pyplot as plt
# Vẽ histogram cho ba thuộc tính Pulse, Exercise, Piercings
plt.figure(figsize=(12, 6))

plt.subplot(1, 3, 1)
plt.hist(data['Pulse'].dropna(), bins=10, color='blue', alpha=0.7)
plt.title('Histogram of Pulse')

plt.subplot(1, 3, 2)
plt.hist(data['Exercise'].dropna(), bins=10, color='green', alpha=0.7)
plt.title('Histogram of Exercise')

plt.subplot(1, 3, 3)
plt.hist(data['Piercings'].dropna(), bins=10, color='red', alpha=0.7)
plt.title('Histogram of Piercings')

plt.tight_layout()
plt.show()
```

b. Phân tích phân phối:

- Nhịp tim (Pulse): Có thể lệch trái, lệch phải hoặc đối xứng dựa trên đồ thị.
- Số giờ tập thể thao (Exercise): Thường có thể lệch phải vì phần lớn sinh viên có thể tập ít giờ.
- Số khuyên đeo (Piercings): Thường lệch phải vì không nhiều người có nhiều khuyên.

Câu 4:

a. Lấy mẫu ngẫu nhiên cỡ mẫu 50 và tính tỷ lệ nữ, so sánh với tỷ lệ gốc:

```
# Lấy mẫu ngẫu nhiên với cỡ mẫu là 50
sample_50 = data.sample(50, random_state=42)
female_rate_sample = (sample_50['Gender'] == 'F').mean() * 100
female_rate_total = (data['Gender'] == 'F').mean() * 100
error = abs(female_rate_sample - female_rate_total)

print(f"Tỷ lệ nữ trong mẫu: {female_rate_sample:.2f}%")
print(f"Tỷ lệ nữ trong toàn bộ dữ liệu: {female_rate_total:.2f}%")
print(f"Sai số: {error:.2f}%")
```

b. Thực hiện lại quá trình trên 100 lần và vẽ histogram của các tỷ lệ nữ:

```
# Lặp lại việc lấy mẫu ngẫu nhiên 100 lần
female_rates = []
for i in range(100):
    sample = data.sample(50, random_state=i)
    female_rate = (sample['Gender'] == 'F').mean() * 100
    female_rates.append(female_rate)

# Vẽ histogram của 100 giá trị tỷ lệ nữ
plt.hist(female_rates, bins=10, color='purple', alpha=0.7)
plt.title('Histogram of Female Rates in 100 Samples (n=50)')
plt.xlabel('Female Rate (%)')
plt.ylabel('Frequency')
plt.show()
```

c. Thực hiện với các cỡ mẫu khác nhau:

```
sample_sizes = [30, 50, 100, 200]
for n in sample_sizes:
    female_rates = []
    for i in range(100):
        sample = data.sample(n, random_state=i)
        female_rate = (sample['Gender'] == 'F').mean() * 100
        female_rates.append(female_rate)

    plt.hist(female_rates, bins=10, alpha=0.7, label=f'n={n}')

plt.title('Histogram of Female Rates for Different Sample Sizes')
plt.xlabel('Female Rate (%)')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```

---Kết thúc đề 02 - Bộ dữ liệu StudentSurvey.csv ---

Đề 03 - file bds.csv

Đề bài: KHẢO SÁT DỮ LIỆU VỀ MUA BÁN BẤT ĐỘNG SẢN NĂM 2020 TẠI VIỆT NAM

Dữ liệu được cung cấp trong file *bds.csv* được thu thập năm 2020 tại Việt Nam.

title	poster_temp	area_temp	final_price	type	acreage
Bán nhà tại p.Phạm Ngũ Lão, Quận 1, 25m2, 4.2 Tỷ	môi giới	tp hồ chí minh	4200000000 0	nhà	25

1. (1 điểm) Đọc dữ liệu và hiển thị 10 dòng đầu tiên của dữ liệu

2. (5 điểm) Tìm hiểu thông tin về dữ liệu:

a. (0.75 điểm) Số lượng loại mỗi loại (type) bất động sản có trong dữ liệu

Loại BDS	Số lượng
nhà	
chung cư	
đất	
khác	

b. (2 điểm)

i. (0.5 điểm) Số lượng bất động sản của mỗi loại (type) theo từng cách thức rao bán (poster_temp).

Cách rao bán	khác	môi giới
nhà		
chung cư		
đất		
khác		

ii. (1.5 điểm) Vẽ biểu đồ tròn thể hiện phần trăm của từng loại bất động sản theo từng cách thức rao bán dựa vào bảng trên và đưa ra nhận xét (tối thiểu 2 nhận xét)

c. (1.0 điểm)

i. Tạo bảng dữ liệu mới chỉ có 3 vùng (tp hồ chí minh, hà nội và đà nẵng) và cho biết số dòng của bảng dữ liệu mới

ii. (0.75) Từ bảng dữ liệu trên hãy so sánh các đại lượng thống kê giá trị trung bình, trung vị, độ lệch chuẩn của giá BDS của mỗi vùng, vẽ biểu đồ cột và đưa ra nhận xét

d. (1.25 điểm)

i. (0.5 điểm) Tạo bảng dữ liệu có diện tích (acreage khác -1) và cho biết số dòng của bảng dữ liệu mới.

ii. (1.0 điểm) Vẽ biểu đồ thể hiện số BDS có diện tích trong mô tả (acreage khác -1) của mỗi loại poster và đưa ra nhận xét

3. (2 điểm)

a. (1 điểm) Chuyển dữ liệu time thành kiểu datetime, vẽ biểu đồ thể hiện số lượng bất động sản được bán theo từng tháng trong năm 2020 và đưa ra nhận xét

b. (1 điểm) Hãy tính xác suất để một bất động sản được bán trong mùa hè (từ tháng 6 đến tháng 8) năm 2020

4. (2 điểm) Hãy dựa vào bảng dữ liệu được cung cấp, tính xác suất để trong 50 bất động sản được bán có 25 bất động sản có giá trị lớn hơn 3 tỷ

Câu 1: Đọc dữ liệu và hiển thị 10 dòng đầu tiên

```
import pandas as pd
# Đọc dữ liệu từ file CSV
data = pd.read_csv('bds.csv')

# Hiển thị 10 dòng đầu tiên
print(data.head(10))
```

Câu 2: Tìm hiểu thông tin về dữ liệu

a. Số lượng loại bất động sản (type)

```
# Đếm số lượng mỗi loại bất động sản
type_counts = data['type'].value_counts()
print(type_counts)
```

```
type
nhà      147416
khác     63968
đất      61352
chung cư   9909
Name: count, dtype: int64
```

b. Số lượng bất động sản theo cách rao bán

i. Số lượng theo từng loại và cách thức rao bán

```
# Đếm số lượng bất động sản theo type và poster_temp
poster_type_counts = data.groupby(['type',
                                   'poster_temp']).size().unstack(fill_value=0)
print(poster_type_counts)
```

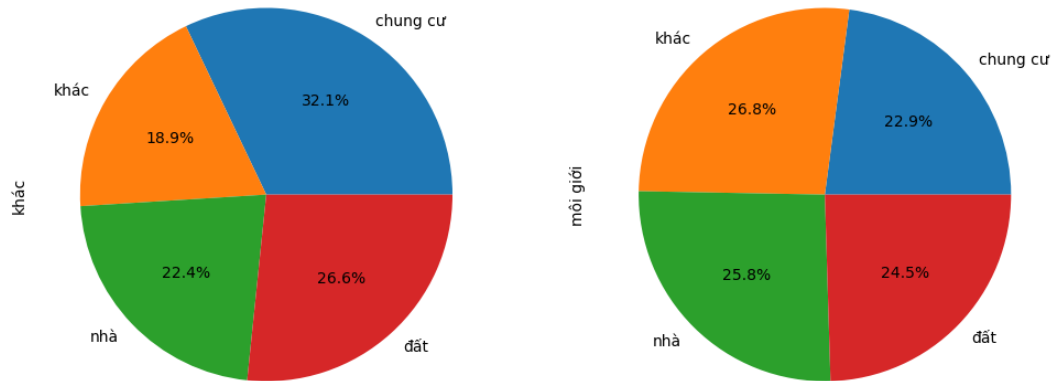
```
poster_temp  khác  môi giới
type
chung cư      2903      7006
khác          11024     52944
nhà           30096     117320
đất           14914     46438
```

ii. Vẽ biểu đồ tròn phần trăm từng loại bất động sản theo cách rao bán

```
import matplotlib.pyplot as plt

# Tính phần trăm từng loại bất động sản
poster_type_percent =
poster_type_counts.div(poster_type_counts.sum(axis=1), axis=0) * 100
```

```
# Vẽ biểu đồ tròn cho mỗi loại
poster_type_percent.plot(kind='pie', subplots=True, figsize=(12, 6),
autopct='%1.1f%%', legend=False)
plt.show()
```



c. Bảng dữ liệu mới chỉ có 3 vùng (TP HCM, Hà Nội, Đà Nẵng)

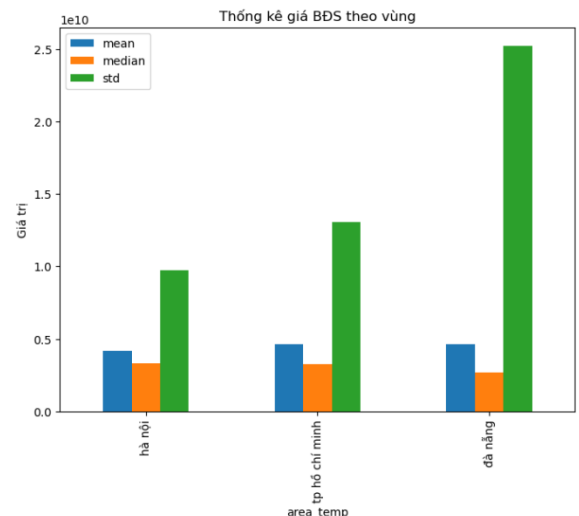
i. Tạo bảng dữ liệu mới và đếm số dòng

```
# Lọc dữ liệu chỉ với 3 vùng
filtered_data = data[data['area_temp'].isin(['tp hồ chí minh', 'hà
nội', 'đà nẵng'])]
print(f"Số dòng dữ liệu mới: {len(filtered_data)}")
```

ii. So sánh các đại lượng thống kê về giá trị BDS

```
# Thống kê giá trị trung bình, trung vị và độ lệch chuẩn của giá trị BDS
price_stats = filtered_data.groupby('area_temp')['final_price'].agg(['mean',
'median', 'std'])
print(price_stats)
```

```
# Vẽ biểu đồ cột
price_stats.plot(kind='bar', figsize=(8, 6))
plt.title('Thống kê giá BDS theo vùng')
plt.ylabel('Giá trị')
plt.show()
```



d. Diện tích BDS khác -1

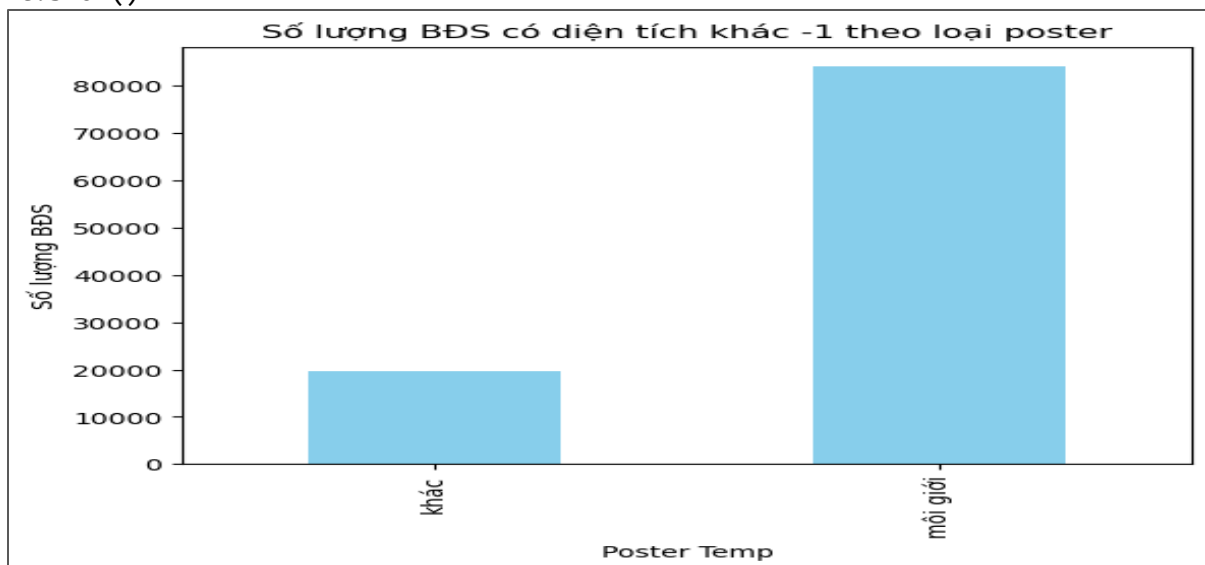
i. Tạo bảng dữ liệu diện tích khác -1

```
# Lọc dữ liệu có diện tích khác -1
filtered_area_data = data[data['acreage'] != -1]
print(f"Số dòng dữ liệu mới: {len(filtered_area_data)}")
```

ii. Vẽ biểu đồ về số BDS theo diện tích và loại poster

```
# Đếm số BDS theo diện tích và loại poster
area_poster_counts = filtered_area_data.groupby('poster_temp')['acreage'].size()

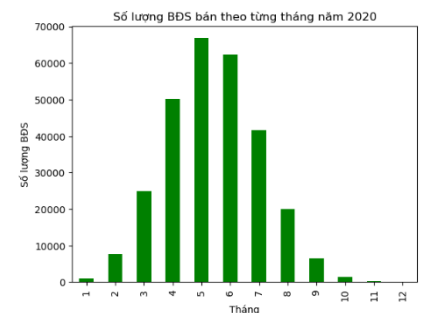
# Vẽ biểu đồ
area_poster_counts.plot(kind='bar', color='skyblue')
plt.title('Số lượng BDS có diện tích khác -1 theo loại poster')
plt.xlabel('Poster Temp')
plt.ylabel('Số lượng BDS')
plt.show()
```



Câu 3: Phân tích thời gian

a. Chuyển cột time thành kiểu datetime và vẽ biểu đồ

```
# Chuyển cột time thành kiểu datetime
data['time'] = pd.to_datetime(data['time'])
# Nhóm theo tháng và đếm số lượng BDS
data['month'] = data['time'].dt.month
monthly_sales = data.groupby('month').size()
# Vẽ biểu đồ số lượng BDS theo từng tháng
monthly_sales.plot(kind='bar', color='green')
plt.title('Số lượng BDS bán theo từng tháng năm 2020')
plt.xlabel('Tháng')
plt.ylabel('Số lượng BDS')
plt.show()
```



b. Tính xác suất bán trong mùa hè

```
# Lọc dữ liệu bán trong mùa hè (tháng 6-8)
summer_sales = data[(data['month'] >= 6) & (data['month'] <= 8)].shape[0]

# Tính xác suất
summer_probability = summer_sales / data.shape[0]
print(f"Xác suất bán BĐS trong mùa hè: {summer_probability:.2%}")
```

Câu 4: Xác suất BĐS có giá trị lớn hơn 3 tỷ

```
import scipy.stats as stats

# Tính xác suất một BĐS có giá trị lớn hơn 3 tỷ
prob_greater_than_3_billion = (data['final_price'] > 3e9).mean()

# Xác suất để 25 trong 50 BĐS có giá trị lớn hơn 3 tỷ
n = 50
k = 25
binom_prob = stats.binom.pmf(k, n, prob_greater_than_3_billion)
print(f"Xác suất 25/50 BĐS có giá trị lớn hơn 3 tỷ: {binom_prob:.4f}")
```

---Kết thúc Đề 03 - file bds.csv ---

1. THỐNG KÊ MÔ TẢ (DESCRIPTIVE STATISTICS)

Giá trị trung bình (Mean): Đại diện cho giá trị trung bình cộng của một tập hợp dữ liệu.

```
import numpy as np
data = [2, 4, 6, 8, 10]
mean = np.mean(data)
print("Mean:", mean)
```

Trung vị (Median): Là giá trị giữa của dữ liệu khi đã được sắp xếp.

```
median = np.median(data)
print("Median:", median)
```

Độ lệch chuẩn (Standard Deviation): Đo lường sự phân tán của dữ liệu so với giá trị trung bình.

```
std_dev = np.std(data)
print("Standard Deviation:", std_dev)
```

Trực quan hóa dữ liệu: Các biểu đồ thường dùng bao gồm:

- **Histogram:** Để hiểu phân bố của dữ liệu.
- **Boxplot:** Để hiển thị các đặc trưng như trung vị, phần trăm các giá trị.

```
import matplotlib.pyplot as plt
plt.hist(data)
plt.show()
```

2. MÔ PHỎNG (SIMULATION)

Mô phỏng là quá trình tạo ra dữ liệu ngẫu nhiên để mô phỏng một quá trình hoặc sự kiện thực tế, thường dùng trong các bài toán dự đoán và nghiên cứu xác suất. Mô phỏng biến ngẫu nhiên: Dùng numpy để mô phỏng các phân phối ngẫu nhiên.

Mô phỏng tung đồng xu (2 giá trị xác suất: mặt ngửa hoặc mặt sấp):

```
outcomes = np.random.choice(['Heads', 'Tails'], size=10)
print(outcomes)
```

Mô phỏng Lăn xúc xắc (6 giá trị xác suất):

```
dice_rolls = np.random.choice([1, 2, 3, 4, 5, 6], size=10)
print(dice_rolls)
```

Mô phỏng chuỗi thời gian (Time Series):

```
time_series = np.cumsum(np.random.randn(1000))
plt.plot(time_series)
plt.show()
```

3. Phân phối xác suất (Probability Distributions)

Phân phối xác suất mô tả khả năng xuất hiện của các kết quả khác nhau trong một không gian mẫu.

Phân phối chuẩn (Normal Distribution): Đường cong hình chuông, đặc trưng bởi giá trị trung bình và độ lệch chuẩn.

```
from scipy.stats import norm
x = np.linspace(-5, 5, 100)
plt.plot(x, norm.pdf(x, 0, 1)) # Mean=0, StdDev=1
plt.title("Normal Distribution")
plt.show()
```

Phân phối nhị thức (Binomial Distribution): Số lần thành công trong một số lần thử cố định, với xác suất thành công cố định.

```
from scipy.stats import binom
n, p = 10, 0.5 # 10 trials, probability of success = 0.5
x = np.arange(0, n+1)
plt.plot(x, binom.pmf(x, n, p))
plt.title("Binomial Distribution")
plt.show()
```

Phân phối Poisson: Dùng để mô tả số lần một sự kiện xảy ra trong một khoảng thời gian xác định.

```
from scipy.stats import poisson
plt.plot(x, poisson.pmf(x, mu=3))
plt.title("Poisson Distribution")
plt.show()
```

Xem lại các bài lab đã thực hành

NUMPY

```
# Phát sinh ngẫu nhiên mảng arr gồm 100 phần tử số nguyên có giá trị từ 0 đến 1000
```

```
arr = np.random.randint(0, 1001, 100)
print("Mảng ngẫu nhiên arr =\n",arr)
```

```
#Câu 1
```

```
# 1. Hiển thị 10 phần tử đầu tiên
```

```
print("10 phần tử đầu tiên:", arr[:10])
```

```
# 2. Hiển thị 5 phần tử cuối cùng
```

```
print("5 phần tử cuối cùng:", arr[-5:])
```

```
# 3. Hiển thị các phần tử thỏa mãn điều kiện chia hết cho 5 và lớn hơn 100
```

```
result = arr[(arr % 5 == 0) & (arr > 100)]
```

```
print("Các phần tử chia hết cho 5 và lớn hơn 100:", result)
```

```
#Tính tổng mảng
```

```
print(f'- Tổng: {arr.sum()}')
```

```
#Tìm giá trị trung bình
```

```
print('- Trung bình:', arr.mean())
```

```
#Tìm giá trị trung vị
```

```
print('- Trung vị:', np.median(arr))
```

```
#Tìm Q1, Q2, Q3
```

```
print('- Tứ phân vị [Q1, Q2, Q3]:', np.quantile(arr, [0.25, 0.5, 0.75]))
```

```
#Tìm phân vị 20 (percentile 20)
```

```
print('- Phân vị 20:', np.percentile(arr, 20))
```

```
import numpy as np
```

```
# Chọn ngẫu nhiên 10 phần tử không hoàn lại từ mảng arr 5 lần
```

```
for i in range(5):
```

```
    selected = np.random.choice(arr, size=10, replace=False)
```

```
    print(f'Lần chọn ngẫu nhiên {i+1}: {selected}')
```

MATPLOTLIB

Câu 6: Vẽ đồ thị cột

```
import numpy as np
import matplotlib.pyplot as plt

# Vẽ đồ thị cột đơn giản
x = np.array(["Java", "Python", "PHP", "JavaScript", "C#", "C++"])
y = np.array([22, 18, 9, 8, 7.7, 7])

# Thiết lập tiêu đề và nhãn cho các trục
plt.title('Mức độ phổ biến của các ngôn ngữ lập trình')
plt.xlabel('Programming Language')
plt.ylabel('Poplarity')

plt.bar(x,y)
plt.show()
```

Câu 7: Vẽ đồ thị bánh pie

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([31.27, 24.79, 12.39, 11.27, 10.85, 9.44])
mylabels = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
myexplode = [0.1, 0, 0, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode,
        shadow=True, startangle=-90, autopct='%0.2f%%')

plt.axis('equal') # Đảm bảo biểu đồ là hình tròn
plt.show()
```

#startangle là điều chỉnh góc độ thê độ 180 360

#shadow: là bóng

#explode: bôi đậm phần tham số đổ

#autopct: hiển thị tỉ lệ phần trăm

```

# Câu 8: vẽ histogram
import numpy as np
import matplotlib.pyplot as plt

# Phát sinh 10000 số ngẫu nhiên thuộc phân phối chuẩn
data = np.random.normal(size=10000)

# Vẽ histogram của các số ngẫu nhiên #Vẽ histogram sử dụng hàm hist()
plt.hist(data, bins=100) #bins: chia thành 100 cột

# Thêm tiêu đề và nhãn
plt.title('Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')

# Hiển thị biểu đồ
plt.show()

```

SEABORN

```

import matplotlib.pyplot as plt
import seaborn as sns

# Tải dữ liệu tips
df = sns.load_dataset("tips")

# Vẽ đồ thị thể hiện tổng tiền tip theo ngày
sns.barplot(x="day", y="tip", data=df, estimator=sum, errorbar=None)

# Định dạng và hiển thị đồ thị
plt.title('Tổng tiền tip theo ngày')
plt.xlabel('Ngày')
plt.ylabel('Tổng tiền tip')
plt.show()

# x="day": Trục x biểu diễn các ngày trong tuần.
# y="tip": Trục y biểu diễn giá trị tiền tip.
# data=df: Dữ liệu được sử dụng là DataFrame df.
# estimator=sum: Sử dụng hàm sum để tính tổng tiền tip cho mỗi ngày.
# ci=None: Loại bỏ khoảng tin cậy (confidence interval) cho biểu đồ thanh.
# errorbar=None: nếu không hỗ trợ ci=None thì dùng errorbar để thay thế

```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Tải dữ liệu tips
df = sns.load_dataset("tips")

# Vẽ đồ thị thể hiện tổng tiền tip theo ngày
sns.barplot(x="time", y="tip", data=df, estimator=sum, errorbar=None)

# Định dạng và hiển thị đồ thị
plt.title('Tổng tiền tip theo bữa')
plt.xlabel('Bữa')
plt.ylabel('Tổng tiền tip')
plt.show()
```

```
import seaborn as sns
import matplotlib.pyplot as plt

# Tải dữ liệu tips
df = sns.load_dataset("tips")

# Vẽ boxplot cho cột total_bill
sns.boxplot(x="day", y="total_bill", data=df)

# Định dạng và hiển thị biểu đồ
plt.title('Đồ thị boxplot so sánh tổng hóa đơn giữa các ngày trong tuần')
plt.xlabel('Ngày trong tuần')
plt.ylabel('Tổng tiền hóa đơn')
plt.show()
```

```

import seaborn as sns
import matplotlib.pyplot as plt

# Tải dữ liệu tips
df = sns.load_dataset("tips")

# Tạo subplot để hiển thị hai đồ thị histogram cạnh nhau
fig, axes = plt.subplots(1, 2, figsize=(14, 6), sharey=True)

# Vẽ histogram cho thời gian dùng bữa "Lunch"
sns.histplot(df[df["time"] == "Lunch"]["total_bill"], kde=True, ax=axes[0],
color='skyblue')
axes[0].set_title('Tổng tiền hóa đơn cho bữa trưa (Lunch)')
axes[0].set_xlabel('Tổng tiền hóa đơn')
axes[0].set_ylabel('Tần suất')

# Vẽ histogram cho thời gian dùng bữa "Dinner"
sns.histplot(df[df["time"] == "Dinner"]["total_bill"], kde=True, ax=axes[1],
color='salmon')
axes[1].set_title('Tổng tiền hóa đơn cho bữa tối (Dinner)')
axes[1].set_xlabel('Tổng tiền hóa đơn')

# Hiển thị biểu đồ
plt.tight_layout()
plt.show()

```

Simulation - Mô phỏng & Dữ liệu

#Cau B thực hiện tung đồng xu =100 lần cho biết số lần xuất hiện của mỗi mặt
`import matplotlib.pyplot as plt`

#đếm số lần xuất hiện của mỗi mặt

```
results = {'H':0, 'T':0}
```

```
for i in range(0,100):
```

```
    item = toss_a_fair_coin()
```

```
    results[item]+=1
```

```
print('Kết quả sau khi tung 100 lần',results)
```

```
faces = list(results.keys())
```

```
counts = list(results.values())
```

```
plt.bar(faces, counts, color=['blue', 'orange'])
```

```
plt.xlabel('Mặt đồng xu')
```

```
plt.ylabel('Số lần xuất hiện')
```

```
plt.title(f'Kết quả tung đồng xu 100 lần')
```

```
plt.show()
```

#a. Định nghĩa hàm `toss_a_biased_coin`

```
def toss_a_biased_coin(head_prob):
```

```
    return random.choices(['H', 'T'], weights=[head_prob, 1 - head_prob])[0]
```

#b. Thực hiện việc tung đồng xu n = 100 lần

```
import matplotlib.pyplot as plt
```

Đếm số lần xuất hiện của mỗi mặt

```
def simulate_biased_coin_tosses(num_tosses, head_prob):
```

```
    results = {'H': 0, 'T': 0}
```

```
    for _ in range(num_tosses):
```

```
        result = toss_a_biased_coin(head_prob)
```

```
        results[result] += 1
```

```
    return results
```



```

import random
import matplotlib.pyplot as plt

# a. Phát sinh ngẫu nhiên chiều cao của 1000 người
num_people = 1000
heights = [random.randint(140, 210) for _ in range(num_people)]

# b. Xác định tỷ lệ các mức chiều cao
def categorize_height(height):
    if height < 160:
        return 'Thấp'
    elif 160 <= height < 175:
        return 'Bình thường'
    elif 175 <= height < 190:
        return 'Cao'
    else:
        return 'Rất cao'

# Phân loại các chiều cao
categories = [categorize_height(height) for height in heights]

# Tính tỷ lệ phần trăm cho mỗi mức
category_counts = {category: categories.count(category) for category in ['Thấp',
'Bình thường', 'Cao', 'Rất cao']}
category_percentages = {category: (count / num_people) * 100 for category, count
in category_counts.items()}

# c. Vẽ đồ thị biểu diễn tỷ lệ của mỗi mức chiều cao

# Biểu đồ tròn
plt.figure(figsize=(10, 7))
plt.pie(category_percentages.values(), labels=category_percentages.keys(),
autopct='%1.1f%%', colors=['#ff9999', '#66b3ff', '#99ff99', '#ffcc99'])
plt.title('Tỷ lệ từng mức chiều cao')
plt.show()

# Biểu đồ cột
plt.figure(figsize=(10, 7))
plt.bar(category_percentages.keys(), category_percentages.values(),
color=['#ff9999', '#66b3ff', '#99ff99', '#ffcc99'])
plt.xlabel('Mức chiều cao')
plt.ylabel('Tỷ lệ (%)')
plt.title('Tỷ lệ từng mức chiều cao')
plt.show()

```

DATA – DỮ LIỆU

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Đọc dữ liệu từ tập tin temperature.csv
data = pd.read_csv('temperature.csv')

# Xem qua dữ liệu để hiểu cấu trúc
print(data.head())

# a. Tính toán thống kê cho mỗi tỉnh thành
provinces = data.columns[1:] # Giả sử cột đầu tiên là ngày/tháng và các cột còn
                               # lại là nhiệt độ của các tỉnh

stats = {}

for province in provinces:
    temperatures = data[province].to_numpy()

    min_temp = np.min(temperatures)
    max_temp = np.max(temperatures)
    mean_temp = np.mean(temperatures)
    std_dev = np.std(temperatures)
    temp_range = max_temp - min_temp

    stats[province] = {
        'Min': min_temp,
        'Max': max_temp,
        'Mean': mean_temp,
        'Std Dev': std_dev,
        'Range': temp_range
    }

    print(f"\n{province}:")
    print(f"Nhiệt độ thấp nhất: {min_temp:.2f}")
    print(f"Nhiệt độ cao nhất: {max_temp:.2f}")
    print(f"Nhiệt độ trung bình: {mean_temp:.2f}")
    print(f"Độ lệch chuẩn: {std_dev:.2f}")
    print(f"Độ biến thiên về nhiệt độ: {temp_range:.2f}")
```

```

import pandas as pd
import matplotlib.pyplot as plt
# b. Vẽ histogram nhiệt độ của TP.HCM

# b. Vẽ histogram nhiệt độ của TP.HCM
plt.figure(figsize=(10, 6))

# Trích xuất dữ liệu nhiệt độ của TP.HCM, loại bỏ giá trị NaN
temperature_HCM = data['Ho Chi Minh']

# Vẽ histogram
plt.hist(temperature_HCM, bins=20, edgecolor='black', alpha=0.75)

# Thêm các nhãn và tiêu đề
plt.xlabel('Nhiệt độ (°C)')
plt.ylabel('Số lần xuất hiện')
plt.title('Histogram nhiệt độ của TP.HCM')
plt.grid(True)

# Hiển thị đồ thị
plt.show()

```

```

#c. Lựa chọn đồ thị để so sánh nhiệt độ của 6 tỉnh thành
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Chọn 6 tỉnh thành
selected_provinces = ['Ha Noi', 'Vinh', 'Da Nang', 'Nha Trang', 'Ho Chi Minh', 'Ca Mau']

# Tính toán nhiệt độ trung bình của mỗi tỉnh thành
mean_temps = [data[province].dropna().mean() for province in selected_provinces]

# Vẽ biểu đồ cột
plt.figure(figsize=(12, 6))
plt.bar(selected_provinces, mean_temps,
color=['#ff9999', '#66b3ff', '#99ff99', '#ffcc99', '#ffb3e6', '#c2c2f0'])
plt.xlabel('Tỉnh thành')
plt.ylabel('Nhiệt độ trung bình (°C)')
plt.title('So sánh nhiệt độ trung bình của 6 tỉnh thành')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```

```

# a. Tỷ lệ nam nữ
genders = data['Gender']
unique, counts = np.unique(genders, return_counts=True)
gender_counts = dict(zip(unique, counts))
total_count = len(genders)
gender_ratios = {gender: count / total_count for gender, count in
gender_counts.items()}

print("Tỷ lệ nam nữ:")
for gender, ratio in gender_ratios.items():
    print(f"{gender}: {ratio:.2%}")

```

```

# Tính toán các đại lượng thống kê cho chiều cao và cân nặng
heights = data['Height_cm']
weights = data['Weight_kg']

def calculate_statistics(array):
    min_val = np.min(array)
    max_val = np.max(array)
    range_val = max_val - min_val
    mean_val = np.mean(array)
    std_dev = np.std(array)
    return min_val, max_val, range_val, mean_val, std_dev

height_stats = calculate_statistics(heights)
weight_stats = calculate_statistics(weights)

print("\nCác đại lượng thống kê:")
print("Chiều cao (Height):")
print(f"  Min: {height_stats[0]:.2f} cm")
print(f"  Max: {height_stats[1]:.2f} cm")
print(f"  Range: {height_stats[2]:.2f} cm")
print(f"  Mean: {height_stats[3]:.2f} cm")
print(f"  Std Dev: {height_stats[4]:.2f} cm")

print("\nCân nặng (Weight):")
print(f"  Min: {weight_stats[0]:.2f} kg")
print(f"  Max: {weight_stats[1]:.2f} kg")
print(f"  Range: {weight_stats[2]:.2f} kg")
print(f"  Mean: {weight_stats[3]:.2f} kg")
print(f"  Std Dev: {weight_stats[4]:.2f} kg")

```

#b. Vẽ đồ thị histogram về chiều cao và cân nặng của hai thuộc tính chiều cao và cân nặng

```
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.hist(heights, bins=20, edgecolor='black', alpha=0.75)
plt.xlabel('Chiều cao (cm)')
plt.ylabel('Số lần xuất hiện')
plt.title('Histogram chiều cao')

plt.subplot(1, 2, 2)
plt.hist(weights, bins=20, edgecolor='black', alpha=0.75)
plt.xlabel('Cân nặng (kg)')
plt.ylabel('Số lần xuất hiện')
plt.title('Histogram cân nặng')

plt.tight_layout()
plt.show()
```

5. Có bao nhiêu nam, bao nhiêu nữ. Tỷ lệ bao nhiêu?

Số lượng nam, nữ và tỷ lệ

```
gender_counts = df['sex'].value_counts()
num_males = gender_counts['Male']
num_females = gender_counts['Female']
male_ratio = num_males / num_rows * 100
female_ratio = num_females / num_rows * 100

print(f"Số lượng nam: {num_males}, Số lượng nữ: {num_females}")
print(f"Tỷ lệ nam: {male_ratio:.2f}%, Tỷ lệ nữ: {female_ratio:.2f}%")
```

6. Số lượng người hút thuốc và tỷ lệ

```
smoker_count = df['smoker'].value_counts()['Yes']
smoker_ratio = smoker_count / num_rows * 100

print(f"Số lượng người hút thuốc: {smoker_count}")
print(f"Tỷ lệ người hút thuốc: {smoker_ratio:.2f}%")
```

```
# 7. Ngày nào trong tuần khách hàng hay đến dùng bữa nhất. Tỷ lệ bao nhiêu?
day_counts = df['day'].value_counts()
most_frequent_day = day_counts.idxmax()
most_frequent_day_count = day_counts.max()
most_frequent_day_ratio = most_frequent_day_count / num_rows * 100

print(f"Ngày khách hàng đến dùng bữa nhiều nhất: {most_frequent_day}\nTỷ lệ: {most_frequent_day_ratio:.2f}%")
```

```
# 3. Giữa nam và nữ ai tip nhiều tiền hơn, ai tip thường xuyên hơn?
# Tính tiền tip trung bình theo giới tính
average_tip_by_gender = df.groupby('sex')['tip'].mean()
tip_frequency_by_gender = df['sex'].value_counts()

print("Tiền tip trung bình theo giới tính:")
print(average_tip_by_gender)
print("Số lần tip theo giới tính:")
print(tip_frequency_by_gender)
```

TRỰC QUAN DỮ LIỆU

```
# 1. Vẽ đồ thị thể hiện tổng tiền tip theo ngày
total_tip_by_day = tips.groupby('day')['tip'].sum()

plt.figure(figsize=(8, 5))
total_tip_by_day.plot(kind='bar', color='skyblue')
plt.title('Tổng tiền tip theo ngày')
plt.xlabel('Ngày')
plt.ylabel('Tổng tiền tip (USD)')
plt.xticks(rotation=45)
plt.show()
```

```
# 3. Vẽ đồ thị histogram của tổng hoá đơn
plt.figure(figsize=(8,5))
plt.hist(tips['total_bill'], bins=20, color='skyblue', edgecolor='black')
plt.title('Histogram của tổng hoá đơn')
plt.xlabel('Tổng hoá đơn (USD)')
```

```
# Tính tổng hóa đơn theo từng ngày
total_bill_by_day = tips.groupby('day')['total_bill'].sum()

# Vẽ biểu đồ cột
plt.figure(figsize=(8, 5))
total_bill_by_day.plot(kind='bar', color='lightcoral')
plt.title('Tổng chi tiêu theo ngày trong tuần')
plt.xlabel('Ngày')
plt.ylabel('Tổng hoá đơn (USD)')
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.show()
```

Trực quan với seaborn

```
# 1. Vẽ đồ thị thể hiện tổng tiền tip theo ngày
plt.figure(figsize=(8, 5))
sns.barplot(x='day', y='tip', data=tips, estimator=sum)
plt.title('Tổng tiền tip theo ngày')
plt.xlabel('Ngày')
plt.ylabel('Tổng tiền tip (USD)')
plt.show()
```

```
# 3. Vẽ đồ thị histogram của tổng hoá đơn
plt.figure(figsize=(8,5))
sns.histplot(tips['total_bill'], bins=20, kde=True, color='skyblue')
plt.title('Histogram của tổng hoá đơn')
plt.xlabel('Tổng hoá đơn (USD)')
```

```
# 4. Vẽ đồ thị boxplot của tổng hoá đơn
plt.figure(figsize=(8,5))
sns.boxplot(x='total_bill', data=tips, color='skyblue')
plt.title('Boxplot của tổng hoá đơn')
plt.xlabel('Tổng hoá đơn (USD)')
```

LAB06 – Tạo quần thể mô phỏng

Bạn hãy tạo một mẫu gồm 10000 phần tử mô phỏng chiều cao của nam thanh niên có giá trị từ 120cm - 200cm. Bạn hãy lưu kết quả vào biến POP.

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
POP = np.random.uniform(120, 200, 10000)
```

Tính: a. Chiều cao trung bình (kỳ vọng) của quần thể và độ lệch chuẩn về chiều cao của quần thể.

```
print("Chiều cao trung bình của quần thể: ", np.mean(POP))
print("Độ lệch chuẩn về chiều cao của quần thể: ", np.std(POP))
```

b. Tính tỷ lệ người cao trong quần thể, biết rằng thanh niên có chiều cao từ 180 trở lên được gọi là cao.

```
CountOfTallMan = np.sum(POP >= 180)
ratioOfTallMan = CountOfTallMan / len(POP) * 100
print("Tỷ lệ người cao trong quần thể: ", ratioOfTallMan, "%")
```

c. Vẽ histogram về chiều cao của quần thể. Theo bạn quần thể có phân phối chuẩn hay không?

```
plt.figure(figsize=(8,5))
plt.hist(POP, bins=30, color='pink', edgecolor='black', alpha=0.7)
plt.title('Histogram về chiều cao của quần thể')
plt.xlabel('Chiều cao (cm)')
plt.ylabel('Số lượng')
plt.grid(axis='y', alpha=0.75)
```

d. Thử vẽ histogram và đồ thị hàm mật độ của phân phối chuẩn sử dụng tham số loc và scale bằng với kỳ vọng và độ lệch chuẩn của quần thể.

```
plt.figure(figsize=(10, 6))
sns.histplot(POP, bins=30, kde=False, color='skyblue', edgecolor='black',
stat='density', label='Histogram')
```

```
std_dev_height = np.std(POP)
mean_height = np.mean(POP)
```

Tính toán hàm mật độ của phân phối chuẩn

```
x = np.linspace(120, 200, 1000)
pdf = (1 / (std_dev_height * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x -
mean_height) / std_dev_height) ** 2)
```



```
# Vẽ đồ thị hàm mật độ
plt.plot(x, pdf, color='red', label='Hàm mật độ phân phối chuẩn')
plt.title('Histogram và hàm mật độ của phân phối chuẩn')
plt.xlabel('Chiều cao (cm)')
plt.ylabel('Mật độ')
plt.legend()
plt.grid()
plt.show()
```

a. Thực hiện bài 2 100 lần, mỗi lần bạn tính được trung bình mẫu. Vẽ đồ thị histogram cho 100 trung bình mẫu bạn tính được.

```
def PPMEAN(data, sample_size, num_samples):
    sample_means = []
    for _ in range(num_samples):
        sample = np.random.choice(data, size=sample_size, replace=False)
        sample_mean = np.mean(sample)
        sample_means.append(sample_mean)
    return sample_means

# Sử dụng hàm để lấy 100 trung bình mẫu với kích thước mẫu là 20
sample_means_20 = PPMEAN(data=POP, sample_size = 20, num_samples=100)

# Vẽ histogram cho 100 trung bình mẫu
plt.figure(figsize=(8, 5))
sns.histplot(sample_means_20, bins=10, color='skyblue', stat='density',
edgecolor='black')
plt.title('Histogram của 100 trung bình mẫu (cỡ 20)')
plt.xlabel('Trung bình mẫu (cm)')
plt.ylabel('Mật độ')
plt.grid()
plt.show()
```